

ARQUITECTURA Y ORQUESTACION CLOUD

J. J. Chán Cuellar

7690-22-1805 Universidad Mariano Gálvez

Seminario de Tecnologías de Información

jchanc4@miumg.edu.gt

Resumen

El presente trabajo analiza la evolución de las arquitecturas tradicionales hacia el uso de microservicios y tecnologías de orquestación en entornos de nube. Se buscó identificar las principales ventajas y desafíos que presenta este enfoque, considerando aspectos relacionados con la escalabilidad, el mantenimiento, la administración y la seguridad de las aplicaciones. Para ello, se revisaron diferentes fuentes relacionadas con arquitectura de software, microservicios, Kubernetes, la metodología Twelve-Factor App y mecanismos de autorización como OAuth 2.0. A partir del análisis realizado, se identificó que los microservicios permiten dividir las aplicaciones en componentes independientes, facilitando su mantenimiento y escalamiento según las necesidades de cada servicio. Se logró determinar que Kubernetes contribuye a simplificar la administración de estos componentes mediante la automatización del despliegue, monitoreo y escalamiento. Asimismo, se observó que las buenas prácticas de desarrollo y la aplicación de mecanismos de seguridad son necesarias para reducir los riesgos asociados con la comunicación entre múltiples servicios. Se concluyó que la implementación de microservicios puede ofrecer mayor flexibilidad y disponibilidad, siempre que exista una adecuada planificación de la infraestructura y se considere la seguridad desde las primeras etapas del desarrollo.

Palabras claves: microservicios, arquitectura, Kubernetes, contenedores, seguridad.

Desarrollo del tema

El desarrollo de software ha cambiado considerablemente durante los últimos años debido a la necesidad de crear sistemas que puedan adaptarse con mayor facilidad a las demandas de los usuarios y las organizaciones. A medida que las aplicaciones han aumentado en tamaño y complejidad, también ha surgido la necesidad de contar con soluciones más fáciles de mantener, actualizar y escalar. Durante mucho tiempo, una de las formas más utilizadas fue la arquitectura monolítica, en la cual todas las funcionalidades se encuentran integradas dentro de una misma unidad de software. Aunque este enfoque puede ser sencillo al inicio de un proyecto, con el crecimiento de la aplicación pueden aparecer dificultades relacionadas con el mantenimiento, las actualizaciones y el uso de recursos. López & Mayab (2017) señalan que este tipo de arquitectura puede generar problemas cuando se necesita modificar o ampliar el sistema, debido a la dependencia existente entre sus diferentes componentes.

Como respuesta a estas limitaciones, la arquitectura de microservicios ha adquirido una mayor importancia dentro del desarrollo de aplicaciones modernas. Este enfoque propone dividir una aplicación grande en diferentes servicios pequeños e independientes, donde cada uno se encarga de una función específica. Vera-Rivera et al. (2019) explican que los microservicios representan tanto un enfoque arquitectónico como organizativo, ya que permiten estructurar las aplicaciones

mediante servicios independientes que se comunican utilizando API bien definidas y protocolos ligeros. López & Mayab (2017) agregan que cada servicio posee su propia lógica de negocio y puede desplegarse de manera independiente. Esto facilita el mantenimiento, ya que una modificación en un servicio específico no necesariamente requiere cambiar toda la aplicación. Además, permite asignar recursos según las necesidades de cada componente, aumentando únicamente las instancias de aquellos servicios que reciben una mayor cantidad de solicitudes.

Distribuir una aplicación en numerosos servicios también aumenta la complejidad de su administración. Ya no se trata únicamente de ejecutar una aplicación en un servidor, sino de controlar múltiples contenedores, servicios, redes y recursos que deben funcionar correctamente entre sí. En este contexto, las plataformas de orquestación son fundamentales. Hohn & Gupta (2018) describen Kubernetes como un sistema de orquestación de contenedores de código abierto que permite administrar aplicaciones distribuidas en múltiples hosts, automatizando tareas como el despliegue, el monitoreo y el escalamiento. Una de sus principales ventajas es que permite definir el estado deseado de una aplicación y encargarse de mantenerlo. Si un contenedor deja de funcionar, Kubernetes puede reemplazarlo automáticamente y, ante un aumento de solicitudes, puede incrementar las instancias disponibles. También proporciona mecanismos como el balanceo de carga y las actualizaciones progresivas (rolling updates), ayudando a mantener la disponibilidad del sistema.

Para aprovechar correctamente estas tecnologías, las aplicaciones deben desarrollarse siguiendo buenas prácticas. Díaz Vidal et al. (2020) destacan la metodología de la Aplicación de 12 Factores (Twelve-Factor App) como una referencia importante para construir aplicaciones destinadas a entornos distribuidos y de nube. Entre sus principios se encuentra declarar las dependencias de forma explícita, mantener la configuración separada del código mediante variables de entorno y diseñar los procesos para funcionar de manera independiente y sin estado (stateless). Esto facilita que diferentes instancias puedan atender solicitudes sin depender de información almacenada localmente en otra instancia. También destaca el principio de disposability o desechabilidad, que busca que los procesos puedan iniciarse y finalizar correctamente, algo especialmente útil en entornos donde los contenedores pueden crearse, eliminarse o reemplazarse constantemente.

La descentralización de los servicios también plantea nuevos desafíos de seguridad. A diferencia de una aplicación monolítica, una arquitectura de microservicios puede contar con múltiples API y puntos de comunicación, aumentando la superficie de exposición del sistema. López & Mayab (2017) advierten que cada una de estas interfaces puede convertirse en un posible punto de acceso para un atacante, por lo que la seguridad debe considerarse desde el diseño de la arquitectura. En este sentido, mecanismos de autorización como OAuth 2.0 permiten controlar el acceso mediante tokens, evitando la necesidad de compartir constantemente credenciales sensibles entre los diferentes componentes. Cada servicio puede validar las solicitudes recibidas y determinar si cuentan con los permisos correspondientes.

En conjunto, la evolución de las arquitecturas monolíticas hacia los microservicios representa un cambio importante en la manera de diseñar, desplegar, administrar y proteger las aplicaciones. Los microservicios proporcionan mayor flexibilidad y escalabilidad, Kubernetes facilita la administración automatizada de la infraestructura y la metodología Twelve-Factor App aporta principios para desarrollar aplicaciones preparadas para entornos distribuidos. A esto se suma la implementación de mecanismos de seguridad que permitan controlar adecuadamente las comunicaciones y los accesos. La combinación de estos elementos permite construir sistemas más adaptables y preparados para las necesidades actuales, aunque también requiere una planificación adecuada y una mayor disciplina durante el desarrollo y la administración de la infraestructura.

La adopción de microservicios también puede favorecer una mejor organización del proceso de desarrollo. Al trabajar con servicios independientes, diferentes equipos pueden concentrarse en componentes específicos sin depender constantemente de los cambios realizados en otras partes de la aplicación. Esto permite trabajar de forma más ordenada y facilita la incorporación de nuevas funcionalidades. Para obtener estos beneficios es necesario establecer una comunicación clara entre los servicios y mantener una adecuada documentación de las API y de los procesos que intervienen en el sistema.

La capacidad de recuperación ante fallos. En una arquitectura distribuida, un problema en un servicio no necesariamente debe provocar la interrupción completa de la aplicación, siempre que los componentes estén correctamente diseñados y exista una infraestructura capaz de detectar y manejar estos errores. El uso de contenedores y herramientas de orquestación permite reemplazar servicios afectados y mantener disponibles aquellos que continúan funcionando. Esto demuestra que la combinación de una arquitectura bien estructurada con mecanismos de automatización puede contribuir significativamente a mejorar la continuidad y confiabilidad de las aplicaciones.

Observaciones y comentarios

El análisis realizado permitió identificar que los microservicios ofrecen mayor flexibilidad y facilidad para escalar los sistemas. También requieren una buena organización y control de la infraestructura. El uso de Kubernetes y buenas prácticas de desarrollo facilita su administración y estabilidad. Además, la seguridad debe considerarse desde el inicio para evitar riesgos en las comunicaciones entre servicios.

Conclusiones

1. La arquitectura de microservicios permite desarrollar sistemas más flexibles, escalables y fáciles de mantener.
2. Los Kubernetes facilitan la administración y disponibilidad de los servicios mediante la automatización de tareas.
3. Aplicar buenas prácticas como *Twelve-Factor App* ayuda a construir aplicaciones preparadas para entornos distribuidos.
4. La seguridad debe considerarse desde el diseño para proteger las comunicaciones y accesos entre los diferentes servicios.

Bibliografía

- Díaz Vidal, J. C., & Matta López, M. (2020). *Sistema de recomendación automático de servicios Multi-cloud* (Proyecto de Grado II, Universidad Icesi).
- Hohn, A., & Gupta, A. (2018). Primeros pasos con Kubernetes *DZone Refcardz*, 233, 1–8.
- López, D., & Mayab, E. (2017). Arquitectura de Software basada en Microservicios para Desarrollo de Aplicaciones Web. En *Séptima Conferencia de Directores de Tecnología de Información, TICAL 2017* (pp. 1-10). RedCLARA.

Vera-Rivera, F. H., Gaona Cuevas, C. M., & Astudillo, H. (2019). Desarrollo de aplicaciones basadas en microservicios: tendencias y desafíos de investigación. *RISTI - Revista Ibérica de Sistemas e Tecnologías de Informação*, (E23), 107-120.

Wiggins, A. (2017). *The Twelve-Factor App*. <https://12factor.net/>

Repositorio del proyecto

El código fuente y el documento en $\text{\LaTeX}$ se encuentran disponibles en el siguiente repositorio de GitHub:

<https://github.com/jonathanc4chan/FORO-ACADEMICO-5-.git>